PROJECT REPORT

ON

DESIGN AND ANALYSIS OF ALGORITHMS

PROJECT TOPIC

SEARCHING AND SORTING ALGORITHM ANALYZER

Submitted By:

Name: Anshu

UID: 26MCD10027

Section: 26MCD-1

Group:A

SubmittedTo:

Ms. Gurpreet Kaur

(E14273)

University Institute of Computing

ChandigarhUniversity,Gharuan,Mohali

BONAFIDE CERTIFICATE

Certified that this project report entitled **"Searching and Sorting Algorithm Analyzer"** is the bonafide work of **Anshu (UID: 26MCD10027)**, who has carried out the project work under the supervision of **Ms. Gurpreet Kaur**. The content and findings presented herein reflect the work undertaken as part of the academic requirements of the course.

The project demonstrates the practical application of Python programming and algorithm analysis through Linear Search, Binary Search, Bubble Sort, Quick Sort, and Merge Sort. It also focuses on time complexity, execution time measurement and performance comparison of different algorithms.

Project Supervisor:	**Head of Department:**
Ms. Gurpreet Kaur	**Dr. Amanpreet Kaur Sandhu**
(E14273)	**HOD – MCA**

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all the individuals who contributed to the successful completion of my project entitled **"Searching and Sorting Algorithm Analyzer."** This project provided me with an opportunity to enhance my understanding of searching and sorting algorithms and their performance analysis.

I would like to express my sincere appreciation to **Dr. Amanpreet Kaur Sandhu, Head of Department (MCA),** and all the faculty members for their valuable support and guidance.

I am especially thankful to my project supervisor, **Ms. Gurpreet Kaur,** for her valuable guidance, encouragement, and support throughout the completion of this project.

I would also like to thank my teachers and friends for their cooperation and encouragement. Finally, I am grateful to my parents for their continuous support and motivation throughout my academic journey.

Anshu
UID: 26MCD10027

INDEX

S. No.	Contents	Page No.
1	Bonafide Certificate	2
2	Acknowledgement	3
3	Introduction	5
4	Aim	5
5	Objectives	5
6	Software Requirements	5
7	Hardware Requirements	6
8	Algorithms Used	6
9	Project Working	6
10	Using Python	7–9
11	Output	10
12	Time Complexity	10
13	Advantages	10
14	Limitations	11
15	Future Scope	11
16	Learning Outcomes	11
17	Conclusion	11

SEARCHING AND SORTING ALGORITHM ANALYZER

1. Introduction

Searching and sorting are two of the most important operations in computer science. Searching is used to find a particular element from a collection of data, while sorting is used to arrange data in a specific order.

The **Searching and Sorting Algorithm Analyzer** is a simple Python-based mini project that implements and compares different searching and sorting algorithms. The project helps users understand how different algorithms work and how their performance changes with the size of the input data.

The project includes **Linear Search, Binary Search, Bubble Sort, Quick Sort, and Merge Sort**. It also calculates the execution time of the algorithms so that their performance can be compared.

2. Aim

To design and develop a simple Python-based application that implements searching and sorting algorithms and compares their performance.

3. Objectives

- To understand the working of searching algorithms.
- To understand the working of sorting algorithms.
- To implement Linear Search and Binary Search.
- To implement Bubble Sort, Quick Sort, and Merge Sort.
- To calculate the execution time of algorithms.
- To compare the performance of different algorithms.
- To understand the importance of time complexity.

4. Software Requirements

- Python
- Visual Studio Code (VS Code)
- Python Extension
- Python Interpreter
- Matplotlib (for performance graph)

5. Hardware Requirements

- Laptop/Desktop
- Minimum 4 GB RAM
- Keyboard and Mouse

6. Algorithms Used

Searching Algorithms

Linear Search:
Checks each element one by one until the required element is found.

Binary Search:
Searches a sorted list by repeatedly dividing the search space into two halves.

Sorting Algorithms

Bubble Sort:
Repeatedly compares adjacent elements and swaps them if they are in the wrong order.

Quick Sort:
Selects a pivot element and divides the array into smaller parts.

Merge Sort:
Divides the array into smaller parts, sorts them, and then merges them.

7. Project Working

The user enters a list of numbers and selects an operation.

The program provides two main options:

1. Searching
2. Sorting

For searching, the user can select Linear Search or Binary Search.

For sorting, the user can select Bubble Sort, Quick Sort, or Merge Sort.

The program also calculates the execution time of the selected algorithm.

8. Using Python

```
import time
# Linear Search
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

# Binary Search
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1

# Bubble Sort
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

    return arr

# Quick Sort
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[0]

    left = [x for x in arr[1:] if x <= pivot]
    right = [x for x in arr[1:] if x > pivot]

    return quick_sort(left) + [pivot] + quick_sort(right)

# Merge Sort
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
   mid = len(arr) // 2
```

```
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    result = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result
# Main Program
print("SEARCHING AND SORTING ALGORITHM ANALYZER")

data = list(map(int, input("Enter numbers separated by space: ").split()))

print("\nOriginal List:")
print(data)

print("\n1. Searching")
print("2. Sorting")

choice = int(input("Enter your choice: "))

if choice == 1:

    key = int(input("Enter element to search: "))

    print("\n1. Linear Search")
    print("2. Binary Search")

    search_choice = int(input("Choose searching algorithm: "))

    if search_choice == 1:
        start = time.perf_counter()
        result = linear_search(data, key)
        end = time.perf_counter()

        if result != -1:
            print("Element found at index:", result)
        else:
            print("Element not found")

        print("Execution Time:",
              end - start, "seconds")

    elif search_choice == 2:
```

```
        sorted_data = sorted(data)

        start = time.perf_counter()
        result = binary_search(sorted_data, key)
        end = time.perf_counter()

        print("Sorted List:", sorted_data)

        if result != -1:
            print("Element found at index:", result)
        else:
            print("Element not found")

        print("Execution Time:",
              end - start, "seconds")

elif choice == 2:

    print("\n1. Bubble Sort")
    print("2. Quick Sort")
    print("3. Merge Sort")

    sort_choice = int(input("Choose sorting algorithm: "))

    if sort_choice == 1:
        start = time.perf_counter()
        result = bubble_sort(data.copy())
        end = time.perf_counter()

        print("Sorted List:", result)
        print("Execution Time:",
              end - start, "seconds")

    elif sort_choice == 2:
        start = time.perf_counter()
        result = quick_sort(data)
        end = time.perf_counter()

        print("Sorted List:", result)
        print("Execution Time:",
              end - start, "seconds")

    elif sort_choice == 3:
        start = time.perf_counter()
        result = merge_sort(data)
        end = time.perf_counter()

        print("Sorted List:", result)
        print("Execution Time:",
              end - start, "seconds")

    else:
        print("Invalid sorting choice.")
else:
    print("Invalid choice.")
```

9. Output

```
PS C:\Users\Lenovo\AppData\Local\Programs\Microsoft VS Code> (Set-ExecutionPolicy -Scope
Process -ExecutionPolicy RemoteSigned) ; (& C:\Users\Lenovo\numpy-env\Scripts\Activate.ps
1)
(numpy-env) PS C:\Users\Lenovo\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Lenov
o\numpy-env\Scripts\python.exe "c:/Users/Lenovo/daa project.py"
SEARCHING AND SORTING ALGORITHM ANALYZER
Enter numbers separated by space: 45 23 76 56 87 34

Original List:
[45, 23, 76, 56, 87, 34]

1. Searching
2. Sorting
Enter your choice: 2

1. Bubble Sort
2. Quick Sort
3. Merge Sort
Choose sorting algorithm: 1
Sorted List: [23, 34, 45, 56, 76, 87]
Execution Time: 3.5099685192108154e-05 seconds
(numpy-env) PS C:\Users\Lenovo\AppData\Local\Programs\Microsoft VS Code>
```

10. Time Complexity

Algorithm	Best Case	Average Case	Worst Case
Linear Search	O(1)	O(n)	O(n)
Binary Search	O(1)	O(log n)	O(log n)
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$
Merge Sort	O(n log n)	O(n log n)	O(n log n)

11. Advantages

- Simple and easy to use.
- Helps understand searching and sorting algorithms.
- Provides execution time comparison.
- Uses user-input data.
- Useful for understanding algorithm performance.
- Suitable for beginners.

12. Limitations

- The project works mainly with numerical data.
- It is a basic command-line application.
- Execution time can be very small for small datasets.
- It does not include advanced algorithms.

13. Future Scope

- Graphical User Interface (GUI).
- More searching and sorting algorithms.
- Performance graphs.
- Random data generation.
- Larger datasets.
- Data structure visualization.

14. Learning Outcomes

After completing this project, the student will be able to:

- Explain searching and sorting algorithms.
- Implement algorithms using Python.
- Compare different algorithms.
- Understand time complexity.
- Analyze algorithm performance.
- Apply algorithmic concepts to practical problems.

15. Conclusion

The **Searching and Sorting Algorithm Analyzer** provides a simple way to understand and compare important searching and sorting algorithms. The project demonstrates how algorithm selection can affect execution time and performance. It also provides practical knowledge of algorithm analysis and Python programming.